Practice CRUD operations on Mongo. 1. Create two collections -- emp and dept.

```
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-03-17T18:38:26.180+05:30: Access control is not enabled for the data
base. Read and write access to data and configuration is unrestricted
------

test> // Insert into dept collection
| db.dept.insertMany([
|   { _id: 10, dname: "ACCOUNTING", loc: "NEW YORK" },
|   { _id: 20, dname: "RESEARCH", loc: "DALLAS" },
|   { _id: 30, dname: "SALES", loc: "CHICAGO" },
|   { _id: 40, dname: "OPERATIONS", loc: "BOSTON" }
| ]);
|
| // Insert into emp collection
| db.emp.insertMany([
|   { ename: "KING", job: "PRESIDENT", sal: 5000, deptno: 10, comm: 0 },
|   { ename: "BLAKE", job: "MANAGER", sal: 2850, deptno: 30, comm: 0 },
|   { ename: "CLARK", job: "MANAGER", sal: 2450, deptno: 10, comm: 0 },
|   { ename: "SCOTT", job: "ANALYST", sal: 3000, deptno: 20, comm: 0 },
|   { ename: "SMITH", job: "CLERK", sal: 800, deptno: 20, comm: 0 },
|   { ename: "ALLEN", job: "SALESMAN", sal: 1600, deptno: 30, comm: 300 }
| ]);
```

2. Insert the emp and dept records.

```
------
   The server generated these startup warnings when booting
   2026-03-17T18:38:26.180+05:30: Access control is not enabled for the data
base. Read and write access to data and configuration is unrestricted
------

test> // Insert into dept collection
| db.dept.insertMany([
|    { _id: 10, dname: "ACCOUNTING", loc: "NEW YORK" },
|    { _id: 20, dname: "RESEARCH", loc: "DALLAS" },
|    { _id: 30, dname: "SALES", loc: "CHICAGO" },
|    { _id: 40, dname: "OPERATIONS", loc: "BOSTON" }
| ]);
|
| // Insert into emp collection
| db.emp.insertMany([
|    { ename: "KING", job: "PRESIDENT", sal: 5000, deptno: 10, comm: 0 },
|    { ename: "BLAKE", job: "MANAGER", sal: 2850, deptno: 30, comm: 0 },
|    { ename: "CLARK", job: "MANAGER", sal: 2450, deptno: 10, comm: 0 },
|    { ename: "SCOTT", job: "ANALYST", sal: 3000, deptno: 20, comm: 0 },
|    { ename: "SMITH", job: "CLERK", sal: 800, deptno: 20, comm: 0 },
|    { ename: "ALLEN", job: "SALESMAN", sal: 1600, deptno: 30, comm: 300 }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69bc264dff30a819737c2907'),
    '1': ObjectId('69bc264dff30a819737c2908'),
    '2': ObjectId('69bc264dff30a819737c2909'),
    '3': ObjectId('69bc264dff30a819737c290a'),
    '4': ObjectId('69bc264dff30a819737c290b'),
    '5': ObjectId('69bc264dff30a819737c290c')
  }
}
test> |
```

3. Increase salary of dept 10 emps by 100.

```
}
test> db.emp.updateMany(
|    { deptno: 10 },
|    { $inc: { sal: 100 } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
test> |
```

4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000.

```
test> db.emp.find(
|   { sal: { $gt: 1000 } },
|   { ename: 1, job: 1, deptno: 1, sal: 1, _id: 0 }
| ).sort({ deptno: 1, job: 1, sal: -1 });
[
  { ename: 'CLARK', job: 'MANAGER', sal: 2550, deptno: 10 },
  { ename: 'KING', job: 'PRESIDENT', sal: 5100, deptno: 10 },
  { ename: 'SCOTT', job: 'ANALYST', sal: 3000, deptno: 20 },
  { ename: 'BLAKE', job: 'MANAGER', sal: 2850, deptno: 30 },
  { ename: 'ALLEN', job: 'SALESMAN', sal: 1600, deptno: 30 }
]
test> |
```

5. Delete the dept with id 40.

```
test> db.dept.deleteOne({ _id: 40 });
{ acknowledged: true, deletedCount: 1 }
test> |
```

2. Learn grouping operations using Mongo aggregation pipeline

1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT.

```
test> db.dept.deleteOne({ _id: 40 });
{ acknowledged: true, deletedCount: 1 }
test> db.emp.aggregate([
|   { $match: { job: { $ne: "PRESIDENT" } } },
|   {
|     $group: {
|       _id: "$job",
|       totalSal: { $sum: "$sal" },
|       maxSal: { $max: "$sal" },
|       minSal: { $min: "$sal" },
|       avgSal: { $avg: "$sal" }
|     }
|   }
| ]);
[
  { _id: 'CLERK', totalSal: 800, maxSal: 800, minSal: 800, avgSal: 800 },
  {
    _id: 'MANAGER',
    totalSal: 5400,
    maxSal: 2850,
    minSal: 2550,
    avgSal: 2700
  },
  {
    _id: 'ANALYST',
    totalSal: 3000,
    maxSal: 3000,
    minSal: 3000,
    avgSal: 3000
  },
  {
    _id: 'SALESMAN',
    totalSal: 1600,
    maxSal: 1600,
    minSal: 1600,
    avgSal: 1600
  }
]
test> |
```

2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

```
test> db.emp.aggregate([
|     {
|       $lookup: {
|         from: "dept",
|         localField: "deptno",
|         foreignField: "_id",
|         as: "dept_details"
|       }
|     },
|     { $unwind: "$dept_details" }, // Flatten the array from lookup
|     {
|       $project: {
|         ename: 1,
|         deptName: "$dept_details.dname",
|         income: { $add: ["$sal", { $ifNull: ["$comm", 0] }] },
|         _id: 0
|       }
|     },
|     { $sort: { income: 1 } }
| ]);
```